

Ex 1) OOP WRITTEN EXAM - Medome (1)

1. TestFct (sum)

```
void testFct() {  
    vector<int> v1 {4, 2, 1, -4};  
    assert { fct<int>(v1) == 3 };  
    vector<int> v2;  
    try {  
        fct<int>(v2);  
        assert(false);  
    }  
    catch (...) {  
        assert(true);  
    }  
    vector<double> v3 {2, 10.5, 5, -10};  
    assert (fct<double>(v3) == 7.5);  
    vector<string> v4 {"y", "g", "a", "u"};  
    assert (fct<string>(v4) == "ygam");  
}
```

template <typename T> ←
T fct(const vector<T> & v)

if (v.empty())
 throw ...

⊕ obs. that it calculates the sum $\Rightarrow T \text{ sum} = v[0];$

```
template <typename T>  
T fct(const vector<T> & v) {  
    if (v.empty())  
        throw exception{};  
    T sum = v[0];  
    for (int i = 1; i < v.size(); i++) {  
        sum = sum + v[i];  
    }  
    return sum;  
}
```

2. TestFct (max)

```
void testFct() {
```

$\Rightarrow$

```
template <typename T>  
T fct(const vector<T> & v) {  
    if (v.empty())  
        throw ...  
    T max = v[0];  
    for (const T & elem: v)  
        if (elem > max)  
            max = elem;  
    return max;  
}
```


1. Adder

template <typename T>

```
void fct2() {
    Adder<int> add(5);
    add = add + 5 + 2;
    ++add;
    add + 8;
    cout << add.sum() << "\n";
    --add;
    cout << add.sum() << "\n";
    --(--add);
    cout << ...
    try {
        --(--(--add));
    }
    catch (...) {}
    ...
}
```

template <typename T>
class Adder {

private:

vector<T> values;

public:

Adder(T value) {

values.push_back(value);

}

Adder& operator+(T value) {

values.push_back(value);

return *this;

}

Adder& operator++() {

values.push_back(values.back());

return *this;

}

Adder& operator--() {

if (values.size() == 1)

throw ...

values.pop_back();

return *this;

}

T sum() const {

T s = values[0];

for (int i = 1; i < values.size(); i++)

s = s + values[i];

return s;

}

};

Complex

```

void Complex() {
    Complex a{1, 2}, b{1, 2}, c{5, 4}, d{6};
    assert (a.getReal() == 0 && a.getImaginary() == 0);
    assert (c.getImaginary() == 4);
    assert (b == d);
    Complex res1 = c/2;
    cout << res1 << "\n";
    try {
        Complex res2 = b/0;
    } catch (...) {
        assert ...
    }
    Vector<string> v1 { std::vector<string> { "hello", "bye" } };
    v1.printAll(std::cout);
    Vector<string> v2 { ... };
    v2.printAll(std::cout);
}
    
```

0, 0 → default constructor
 → copy constructor

hint that real, imaginary are private members

operator ==

operator /

operator <<

throw exception

template <typename T>

```

class Complex {
private:
    int real;
    int imaginary;
public:
    Complex(): real{0}, imaginary{0} {}
    Complex(int real, int imaginary): real{real}, imaginary{imaginary} {}
    int getReal() const {
        return this->real;
    }
    int getImaginary() const {
        return this->imaginary;
    }
    bool operator==(const Complex& other) const {
        return this->real == other.real &&
            this->imaginary == other.imaginary;
    }
    Complex operator/(int value) const {
        if (value == 0)
            throw ...
        return Complex{ this->real / value, this->imaginary / value };
    }
    friend ostream& operator<<(ostream& out, const Complex& c) {
    }
};
    
```



```
ostream& operator<< (ostream& out, const Complex& c) {
    out << c.real << " + " << c.imaginary << "i";
    return out;
}
```

```
template <typename T>
class Vector {
```

private:

```
    Vector<T> elems;
```

public:

```
    Vector(const Vector<T>& elems): elems {elems} {}
```

```
    void printAll(ostream& out) const {
        for (const auto& elem: elems)
            out << elem << " ";
    }
};
```

1. Smart pointer, vector

```
void function1() {
```

```
    SmartPointer<int> i1 {new int {1}};
```

```
    SmartPointer<int> i2 {new int {2}};
```

```
    SmartPointer<int> i3 {new int {3}};
```

```
    Vector<SmartPointer<int>> v1 {};
```

→ "default" constructor.

```
    v1.add(i1).add(i2).add(i3);
```

→ claiming

```
    for (auto e: v1) {
        cout << *e << " ";
    }
```

→ +& operator+()

```
    SmartPointer<string> s1 {new string {"A"}};
```

```
    sm ---
```

```
    s2 = s1
```

→ copy constructor & int* count

```
    sm ---
```

```
    s3 {new ...};
```

```
    Vector<SmartPointer<string>> v2 {};
```

```
    v2.add(s2).add(s1);
```

```
    try {
```

```
        v2 = v2 - s2;
```

```
        v2 = v2 - s3;
```

```
    }
```

```
    catch (...) {
```

```
        ...
```

```
};
```



```
template <typename T>
class SmartPointer {
```

```
private:
```

```
T* ptr;
```

```
int* count;
```

```
public:
```

```
SmartPointer(T* ptr): ptr{ptr}, count{new int{1}} {}
```

```
SmartPointer(const SmartPointer<T>& other): ptr{other.ptr},
```

```
count{other.count} {
```

```
(*count)++;
```

```
T& operator*() const {
```

```
return *ptr;
```

```
}
```

```
bool operator==(const SmartPointer<T>& other) const {
```

```
return *ptr == *other.ptr;
```

```
}
```

```
};
```

```
template <typename T>
```

```
class Vector {
```

```
private:
```

```
vector<T> elems;
```

```
public:
```

```
Vector() = default;
```

```
Vector<T>& add(const T& elem) {
```

```
elems.push_back(elem);
```

```
return *this;
```

```
}
```

```
typename vector<T>::iterator begin() {
```

```
return elems.begin();
```

```
}
```

```
typename vector<T>::iterator end() {
```

```
return elems.end();
```

```
}
```

```
Vector<T> operator-(const T& elem) const {
```

```
Vector<T> result;
```

```
bool found = false;
```

```
for (const auto& elem: elems) {
```

```
if (!found && e e == elem)
```

```
found = true;
```

```
else
```

```
result.add(current);
```

```
}
```

```
if (!found) throw ...
```

```
};
```

```
return result;
```


1. Smart Pointer, Set.

```
template <typename T>
class SmartPointer {
```

private:

T* ptr;

int* count;

public:

SmartPointer(T* ptr): ptr{ptr}, count{new int{1}} {}

SmartPointer(const SmartPointer<T>& other):

ptr{other.ptr}, count{other.count} {}

{ (*count)++; }

SmartPointer<T>& operator=(const SmartPointer<T>& other){

if (&this == &other)

return *this;

(*count)--;

if (*count == 0){

delete ptr;

delete count;

}

ptr = other.ptr;

count = other.count;

(*count)++;

return *this;

~SmartPointer() {

(*count)--;

if (*count == 0){

delete ptr;

delete count;

}

T& operator*() const {

return *ptr;

}

bool operator==(const SmartPointer<T>& other) const {

return *ptr == *other.ptr;

}

}


```
template <typename T>
class Set {
```

private:

```
vector<T> elems;
```

public:

```
Set() = default;
```

```
Set<T> operator+(const T& elem) const {
```

```
    Set<T> result = *this;
```

```
    for (const auto& current : result.elems)
```

```
        if (current == elem)
```

```
            throw ....
```

```
    result.elems.push_back(elem);
```

```
    return result;
```

```
}
```

```
Set<T>& remove(const T& elem) {
```

```
    for (auto it = elems.begin(); it != elems.end(); it++) {
```

```
        if (*it == elem) {
```

```
            elems.erase(it);
```

```
            return *this;
```

```
        }
```

```
    }
```

```
    return *this;
```

```
}
```

```
}
```

1. Stack

```
template <typename T>
```

```
class Stack {
```

private:

```
vector<T> elems;
```

```
int maxCapacity;
```

public:

```
Stack(int capacity) : maxCapacity{capacity} {}
```

```
int getMaxCapacity() const {
```

```
    return maxCapacity;
```

```
}
```

```
Stack<T> operator+(const T& elem) const {
```

```
    if (elems.size() == maxCapacity)
```

```
        throw ...
```

```
    Stack<T> result = *this;
```

```
    result.elems.push_back(elem);
```

```
    return result;
```

```
}
```



```

T pop() {
    T last = elems.back();
    elems.pop_back();
    return last;
}

```



```

}

```

1. ToDolist

```

class Activity {
private:
    std::string description;
    std::string time;
public:
    Activity(const std::string& description, const ... ) ... {}
    friend ostream& operator<< (ostream& out, const Activity& a) {
        out<< "Activity "<<a.description
            << "will take place at "<< a.time
            << ". ";
        return out;
    }
};

```

```

template <typename T>
class Todo {
private:

```

```

    vector<T> elems;

```

```

public:

```

```

    Todo() = default;

```

```

    Todo<T>& operator += (const T& elem) {

```

```

        elems.push_back(elem);

```

```

        return *this;
    }

```

```

    typename vector<T>::iterator begin() {

```

```

        return elems.begin();
    }

```

```

    typename vector<T>::iterator end() {

```

```

        return elems.end();
    }

```

```

    void reversePrint(ostream& out) const {

```

```

        for(auto it = elems.rbegin(); it != elems.rend(); ++it) {

```

```

            out<<*it<<"\n";
        }
    }
};

```

```

};

```



```
class Double: public Object {  
private:  
    double value;  
public:
```

```
    Double(double value): value {value} {}  
    void print() const override {  
        cout << value << " ";
```

```
};
```


1. character

```
class Object {
```

```
public:
```

```
virtual void print() const = 0;
```

```
virtual ~Object() = default;
```

```
};
```

```
class Character : public Object {
```

```
private:
```

```
char value;
```

```
public:
```

```
Character(char value): value {value} {}
```

```
char operator*() const {
```

```
    return value;
```

```
}
```

```
void print() const override {
```

```
    cout << value << " ";
```

```
}
```

```
};
```

```
class MyObjects {
```

```
private:
```

```
std::vector<Object*> objects;
```

```
public:
```

```
MyObjects(Object* object) {
```

```
    objects.push_back(object);
```

```
}
```

```
MyObjects& addBeginning(Object* object) {
```

```
    objects.insert(objects.begin(), object);
```

```
    return *this;
```

```
}
```

```
int size() const {
```

```
    return objects.size();
```

```
}
```

```
auto begin() {
```

```
    return objects.begin();
```

```
}
```

```
auto end() {
```

```
    return objects.end();
```

```
}
```

```
~MyObjects() {
```

```
    for (auto object : objects) {
```

```
        delete object;
```

```
    }
```

```
};
```


Ex1 - written (again) + generated examples ^{AI}

```
void test()
{
    Stack<int> s{3};
    s = s + 10;
    s = s + 20;
    s = s + 30;
    assert(s.getMaxCap() == 3);
    assert(s.pop() == 30);
    assert(s.pop() == 20);
}
```

```
template <typename T>
class Stack {
private:
    std::vector<T> elems;
    int maxCapacity;

public:
    Stack(int maxCapacity): maxCapacity{maxCapacity} {}
    int getMaxCapacity() const {
        return maxCapacity;
    }
    Stack& operator+(const T& elem) {
        if (elems.size() == maxCapacity)
            throw ...
        elems.push_back(elem);
        return *this;
    }
    T pop() {
        T e = elems.back();
        elems.pop_back();
        return e;
    }
};
```

2.)

```
void test()
{
    Queue<std::string> q{2};
    q.push("ana"); q.push("maria");
    try {
        q.push("ion");
    }
    catch (...) {
        assert(... Queue is full! ...)
    }
    assert(q.pop() == "ana");
    assert(q.pop() == "maria");
}
```

```
template <typename T>
class Queue {
private:
    std::vector<T> elems;
    int capacity;

public:
    Queue(int cap): capacity{cap} {}
    void push(const T& e) {
        if (elems.size() == cap)
            throw ...
        elems.push_back(e);
    }
    T pop() {
        T e = elems.front();
        elems.erase(elems.begin());
        return e;
    }
}
```


3)

```
void test()
{
    ToDo<std::string> todo;
    todo += "learn";
    todo += "sleep";
    for (auto t: todo)
        std::cout << t << "\n";
}
```

begin()
end()
needed!

```
template <typename T>
class ToDo {
private:
    std::vector<T> elems;
public:
    ToDo& operator+= (const T& e) {
        elems.push_back(e);
        return *this;
    }
}
```

4)

```
void test()
{
    Activity a {"exam", "09:00"};
    std::cout << a;
}
```

```
auto begin() {
    return elems.begin();
}
auto end() {
    return elems.end();
}
```

```
class Activity {
private:
    std::string name;
    std::string hour;
public:
    Activity (std::string n, std::string h): n{n}, h{h} {}
    friend std::ostream& operator<< (std::ostream& out,
        const Activity& a) {
        out << "Activity" << a.name << " at " << a.hour;
        return out;
    }
};
```

5)

```
void test()
{
    Bag<int> b;
    b.add(2);
    b.add(2);
    b.add(5);
    assert (b.count(2) == 2);
    assert (b.count(5) == 1);
}
```

```
template <typename T>
class Bag {
private:
    std::vector<T> elems;
public:
    void add (const T& e) {
        elems.push_back(e);
    }
    int count (const T& e) {
        return std::count(
            elems.begin(), elems.end(),
            e);
    }
};
```

```
friend std::ostream& operator (std::ostream& out, const ... )
friend std::ostream& operator (std::ostream& out, ... );
```

```
friend std::ostream& operator (std::ostream& out, ... )
```

```
friend std::ostream& operator (std::ostream& out, const Activity& a);
return out;
```



```
void test()
```

```
{
```

```
    Repository <int, Car> repo;
```

```
    repo.add(1, Car{"Audi", 2020});
```

```
    repo.add(2, Car{"BMW", 2022});
```

```
    assert(repo.findById(1).getName() == "Audi");
```

```
    try {
```

```
        repo.add(1, Car{"Ford", 2019});
```

```
    }
```

```
    catch (std::...) {
```

```
        assert(strcmp(... Duplicate id ...));
```

```
    }
```

```
}
```

order depends on constructors ✓

```
template <typename ID, typename T>
```

```
class Repository {
```

```
private:
```

```
    std::vector<std::pair<ID, T>> elems;
```

```
public:
```

```
    void add(ID id, const T& obj) {
```

```
        auto it = find_if(elems.begin(), elems.end(),
```

```
            [&](const auto& p) {
```

```
                return p.first == id;
```

```
            });
```

```
        if (it != elems.end())
```

```
            throw ...
```

```
        elems.push_back({id, obj});
```

```
    }
```

```
    T findById(ID id) {
```

```
        auto it = find_if(elems.begin(), elems.end(),
```

```
            [&](const auto& p) {
```

```
                return p.first == id;
```

```
            });
```

```
        if (it == elems.end())
```

```
            throw ...
```

```
        return it->second;
```

```
    }
```

```
};
```

3

1


```

3) void test()
{
    ToDo<std::string> todo;
    todo += "learn";
    todo += "sleep";
    for (auto t: todo)
        std::cout << t << "\n";
}

```

begin()
end()
needed!

```

4) void test()
{
    Activity a {"exam", "09:00"};
    std::cout << a;
}

```

```

class Activity {
private:
    std::string name;
    std::string hour;
public:
    Activity (std::string n, std::string h): n{"n"}, h{"h"} {}
    friend std::ostream& operator << (std::ostream& out,
        const Activity& a) {
        out << "Activity" << a.name << " at" << a.hour;
        return out;
    }
};

```

```

5) void test()
{
    Bag<int> b;
    b.add(2);
    b.add(2);
    b.add(5);
    assert (b.count(2) == 2);
    assert (b.count(5) == 1);
}

```

```

friend std::ostream& operator (std::ostream& out, const ...)
friend std::ostream& operator (std::ostream& out, ---);
friend std::ostream& operator (std::ostream& out, ---)

```

```

template <typename T>
class ToDo {
private:
    std::vector<T> elems;
public:
    ToDo& operator += (const T& e) {
        elems.push_back(e);
        return *this;
    }
    auto begin() {
        return elems.begin();
    }
    auto end() {
        return elems.end();
    }
};

```

```

template <typename T>
class Bag {
private:
    std::vector<T> elems;
public:
    void add (const T& e) {
        elems.push_back(e);
    }
    int count (const T& e) {
        return std::count(
            elems.begin(), elems.end(),
            e);
    }
};

```

```

friend std::ostream& operator (std::ostream& out, const Activity& a) {
    return out;
}

```



```
void test()
```

```
{
```

```
    Repository<int, Car> repo;
```

```
    repo.add(1, Car{"Audi", 2020});
```

```
    repo.add(2, Car{"BMW", 2022});
```

```
    assert(repo.findById(1).getName() == "Audi");
```

```
    try {
```

```
        repo.add(1, Car{"Ford", 2019});
```

```
    }
```

```
    catch (std::...) {
```

```
        assert(strcmp(... Duplicate id ...));
```

```
    }
```

```
}
```

order depends on constructor ✓

```
template <typename ID, typename T>
```

```
class Repository {
```

```
private:
```

```
    std::vector<std::pair<ID, T>> elems;
```

```
public:
```

```
    void add(ID id, const T& obj) {
```

```
        auto it = find_if(elems.begin(), elems.end(),
```

```
            [&](const auto& p) {
```

```
                return p.first == id;
```

```
            });
```

```
        if (it != elems.end())
```

```
            throw ...
```

```
        elems.push_back({id, obj});
```

```
    }
```

```
    T findById(ID id) {
```

```
        auto it = find_if(elems.begin(), elems.end(),
```

```
            [&](const auto& p) {
```

```
                return p.first == id;
```

```
            });
```

```
        if (it == elems.end())
```

```
            throw ...
```

```
        return it->second;
```

```
    }
```

```
};
```

3

7)

```
void test() {
    FilteredList<int> list;
    list.add(1);
    list.add(2);
    list.add(3);
    list.add(4);
    auto even = list.filter([&](int x) {
        return x % 2 == 0;
    });
    assert(even.size() == 2);
}
```

template <typename T>

class FilteredList {

private:

std::vector<T> elems;

public:

```
void add(const T& e) {
    elems.push_back(e);
}
```

8)

```
PriorityList<Task> list;
list.add(Task{"low", 1});
list.add(Task{"high", 10});
list.add(Task{"medium", 5});
list.sortByPriority();
assert(list[0].getName() == "high");
```

template <typename T>

class PriorityList {

private:

std::vector<T> elems;

public:

```
void add(const T& e) {
    elems.push_back(e);
}
```

```
void sortByPriority() {
```

```
    std::sort(elems.begin(), elems.end(), ...)
```

```
    T operator[](int i) { return elems[i]; }
```